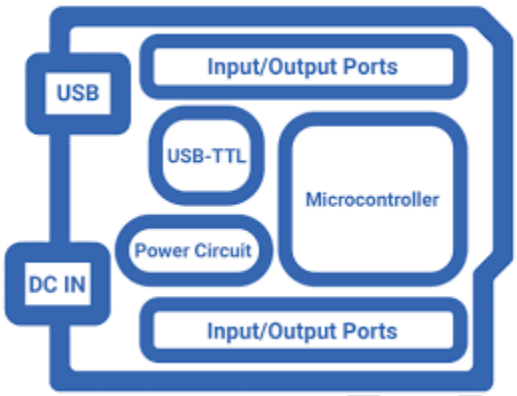
Programming with Arduino Uno

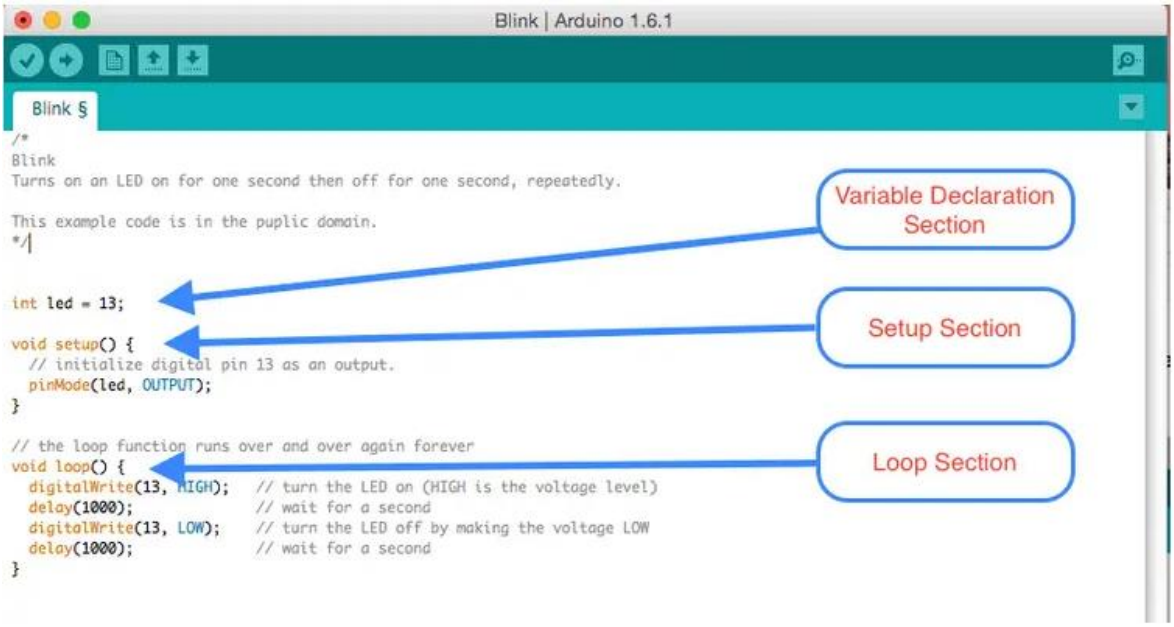
❖ 3.1 ARDUINO UNO board Block diagram



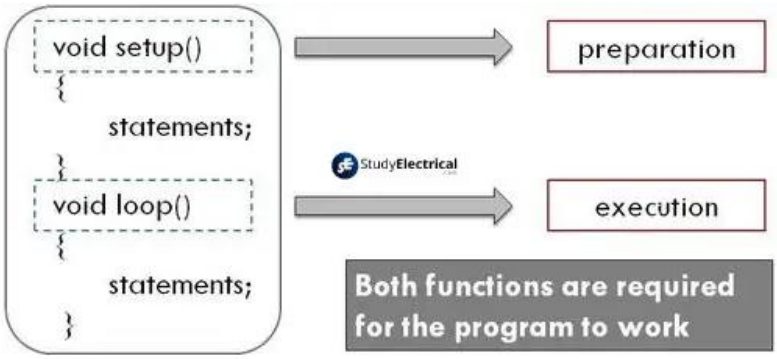
❖ 3.2 Sketch Structure

Arduino programs can be divided in three main parts: Structure, Values (variables and constants), and Functions. In this tutorial, we will learn about the Arduino software program, step by step, and how we can write the program without any syntax or compilation error.
Let us start with the Structure. Software structure consist of two main functions –

- Setup( ) function
- Loop( ) function



These sketches contain the code – the instructions the board will follow. All sketches are divided into two parts – setup function and loop function. You should put code that will run only once (for example, code to set up a board for the application) in the setup function. The loop function contains the code that will be run continuously after the initial setup is complete.



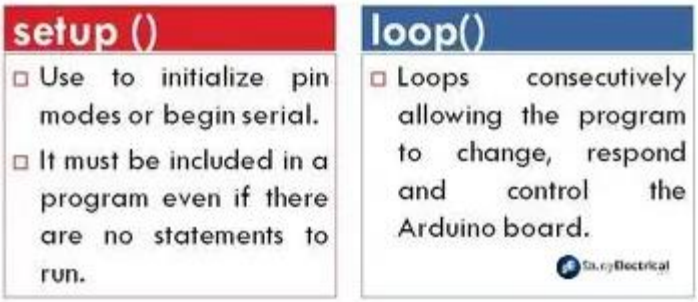
Arduino Program Structure

Here is a typical sketch:

```
const int ledPin = 13; // LED connected to digital pin 13

// The setup() method runs once, when the sketch starts
void setup()
{
pinMode(ledPin, OUTPUT); // initialize the digital pin as an output
}

// the loop() method runs over and over again,
void loop()
{
digitalWrite(ledPin, HIGH); // turn the LED on
delay(1000); // wait a second
digitalWrite(ledPin, LOW); // turn the LED off
delay(1000); // wait a second
}
```



❖ 3.3 Data types & Built in Constants

Arduino data types.They plays an important role when it comes to programming the Arduino. The Arduino which is a computer is highly data agnostic (it does not know or care in what manner the data it receives was sent to it.)

The data types are:

- **void**
- **boolean**
- **char**
- **Unsigned char**
- **byte**
- **int**
- **Unsigned int**
- **Word**
- **Long**
- **Unsigned long**
- **short**
- **float**
- **double**

void

void is only used when declaring functions. It is used to indicate that the function is expected to return no values when they are called.

Example void code

```
void setup()

{

   Serial.begin(9600);

}

void loop(
```

boolean

A boolean holds either one of two boolean values, true or false. boolean is a non-standard type alias for bool defined by Arduino.

This Arduino Data type has a memory of 8 bit / 1 byte.

Example boolean code

```
int LEDpin = 5;       // LED on pin 5

int switchPin = 13;   // momentary switch on 13, other side connected to ground

boolean running = false;

void setup()

{

  pinMode(LEDpin, OUTPUT);

  pinMode(switchPin, INPUT);

  digitalWrite(switchPin, HIGH);      // turn on pullup resistor

}

void loop()

{

  if (digitalRead(switchPin) == LOW)

  {  // switch is pressed - pullup keeps pin high normally

    delay(100);                    // delay to debounce switch

    running = !running;              // toggle running variable

    digitalWrite(LEDpin, running);      // indicate via LED

  }

}
```

char

char which is short for character, is a data type used to store a character value (A,B,C). When initializing multiple characters it will be written in single quotes (eg. ‘A’) but for strings, they use double quotes (eg.”ABC”)

Characters will be stored as numbers. For the specific encoding, you can refer to the ASCII Chart as shown below.

DECIMAL	HEXADECIMAL	CHARACTER	DECIMAL	HEXADECIMAL	CHARACTER
0	0	NULL	32	20	SPACE
1	1	START OF HEADING [SOH]	33	21	!
2	2	START OF TEXT [STX]	34	22	"
3	3	END OF TEXT [ETX]	35	23	#
4	4	END OF TRANSMISSION [EOT]	36	24	$
5	5	ENQUIRY [ENQ]	37	25	%
6	6	ACKNOWLEDGE [ACK]	38	26	&
7	7	BELL [BEL]	39	27	'
8	8	BACKSPACE [BS]	40	28	(
9	9	HARIZONTAL TAB	41	29	)
10	A	LINE FEED	42	2A	*
11	B	VERTICAL TAB [VT]	43	2B	+
12	C	FORM FEED [FF]	44	2C	,
13	D	CARRIAGE RETURN	45	2D	-
14	E	SHIFT OUT [SO]	46	2E	.
15	F	SHIFT IN [SI]	47	2F	/
16	10	DATA LINK ESCAPE [DLE]	48	30	0
17	11	DEVICE CONTROL 1 [DC1]	49	31	1
18	12	DEVICE CONTROL 2 [DC2]	50	32	2
19	13	DEVICE CONTROL 3 [DC3]	51	33	3
20	14	DEVICE CONTROL 4 [DC4]	52	34	4
21	15	NEGATIVE ACKNOWLEDGE [NAK]	53	35	5
22	16	SYNCHRONOUS IDLE [SYN]	54	36	6
23	17	END OF TRANS. BLOCK [ETB]	55	37	7
24	18	CANCEL [CAN]	56	38	8
25	19	END OF MEDIUM [EM]	57	39	9
26	1A	SUBSTITUTE [SUB]	58	3A	:
27	1B	ESCAPE [ESC]	59	3B	;
28	1C	FILE SEPARATOR [FS]	60	3C	<
29	1D	GROUP SEPERATOR [GS]	61	3D	=
30	1E	RECORD SEPERATOR [RS]	62	3E	>
31	1F	UNIT SEPERATOR [US]	63	3F	?

Example char code

char myChar = 'A';

char myChar = 65; // both are equivalent

Unsigned char

The unsigned char datatype encodes numbers from 0 to 255.

This Arduino data type has a memory of 8 bit/ 1 byte which is similar to the byte datatype. For clarity and consistency of the Arduino programming style, for an unsigned, one-byte data type, the byte data type is recommended.

Example Unsigned char code

unsigned char myChar = 240;

byte

Similar to the unsigned char data type, a byte encodes an 8-bit unsigned number from 0-255

Example byte code

byte m = 25 ;//declaration of variable with type byte and initialize it with 25

int

int which is short for integer is one of the most commonly used data type in Arduino. They are your primary data type for storing numbers.

Do note that int size varies from board to board. For example, in ATmega based Arduino boards like the Uno, Mega and Nano, an int uses 2 byte of memory and as a range of -32,768 to +32,767. While for the Due and SAMD based boards (eg. MKR1000, Zero), int uses 4 byte of memory and as a range of -2,147,483,648 to + 2,147,483,647.

Example int code

int countUp = 0; //creates a variable integer called 'countUp'

Unsigned int

The unsigned int is similar to int in the way that they store a 2-byte value. However, instead of storing negative numbers, they store only positive values with a range of 0 to +65,535.

Same as int, unsigned int size varies from board to board with ATmega based Arduino boards storing a 2-byte value while the Due and SAMD based boards stores a 4 bytes (32-bit) value and has a range of 0 to 4,294,967,295.

The main difference between unsigned int and ints is how the highest bit/sign bit is interpreted. In the int type (which is signed), if the highest bit is “1”, the number is interpreted as a negative number, and the other 15 bits are interpreted with.

Example unsigned int code

unsigned int ledPin = 13;

Word

The word data type is very similar to the previous unsigned int data type. On the ATmega based Arduino boards, a word stores a 16-bit unsigned number with a 2-byte value and a range from 0 to +65535. As for Due and SAMD based boards, it stores a 32-bit unsigned number with a 4-byte value.

Example Word code

word w = 10000;

Long

Long variables are extended size variables for number storage. Long variables use 4 bytes from memory (32 bits) with a range from -2,147,483,648 to +2,147,483,647.

Example Long code

long speedOfLight = 186000L; //declaration of variable with type Long and initialize it with 186000

Unsigned long

Similar to the Long data type, unsigned long variables are extended size variables for number storage and use 4 bytes from memory (32 bits). However, unlike standard Longs, unsigned longs do not store negative numbers. They have a range of 0 to +4,294,967,295.

Example Unsigned long code

```
unsigned long time;

void setup() {

  Serial.begin(9600);

}

void loop() {

  Serial.print("Time: ");

  time = millis();

  //prints time since program started

  Serial.println(time);

  // wait a second so as not to send massive amounts of data

  delay(1000);

}
```

short

A short datatype stores a 16 bit value and uses 2 bytes from memory on ALL Arduinos. They have a range of -32,768 to +32,767.

Example short code

```
short ledPin = 13
```

float

The float is one of the most important Arduino data type as it can store decimal numbers. This data type is for floating-point numbers which are numbers with a decimal point.

Floating-point numbers are often used to approximate the analog and continuous values because they have greater resolution than integers.

This data type has a memory of 32 bit/ 4 bytes and a range of -3.4028235E+38 to +3.4028235E+38.

Example float code

```
float myfloat;

float sensorCalbrate = 1.117;

int x;

int y;

float z;

x = 1;

y = x / 2;          // y now contains 0, ints can't hold fractions

z = (float)x / 2.0; // z now contains .5 (you have to use 2.0, not 2)
```

double

This data type is a double-precision floating point number. On ATmega based Arduino boards like the Uno, Mega and Nano, double precision floating-point number occupies 4 bytes (32 bit). That is, the double implementation is exactly the same as the float, with no gain in precision.

While for Due and SAMD based boards (eg. MKR1000, Zero), double have 8 bytes (64-bit) precision.

Example double code

```
double num = 45.352 ;// declaration of variable with type double and initialize it with 45.352
```

❖ 3.4 Operators: Arithmetic, Bitwise, Compound, Comparison, and Boolean

An operator is a symbol that tells the compiler to perform specific mathematical or logical functions. C language is rich in built-in operators and provides the following types of operators

- The types of Operators classified in Arduino are:

1. Arithmetic Operators
2. Compound Operators
3. Boolean Operators
4. Comparison Operators
5. Bitwise Operators

1.Arithmetic Operators

There are six basic operators responsible for performing mathematical operations in Arduino, which are listed below:

- Assignment Operator (=)

The Assignment operator in Arduino is used to set the variable's value. It is quite different from the equal symbol (=) normally used in mathematics.

- Addition (+)

The addition operator is used for the addition of two numbers. For example, P + Q.

- Subtraction (-)

Subtraction is used to subtract one value from the another. For example, P - Q.

- Multiplication (*)

The multiplication is used to multiply two numbers. For example, P * Q.

- Division (/)

The division is used to determine the result of one number divided with another. For example, P/Q.

- Modulo (%)

The Modulo operator is used to calculate the remainder after the division of one number by another number.

Most of the operators are similar to the usual operator used in mathematics.

Let's understand the operators with the help of two examples.

Example 1:

Consider the below code.

```
int b;

void setup ( )

{

Serial.begin( 9600 );

}

void loop ( )

{

b = 5 + 2;

Serial.println(b);

}
```

2. Compound Operators

- The compound operators perform two or more calculations at once.
- The result of the right operand is assigned to the left operand, as already discussed above. The same condition will apply to all the compound operators, which are listed below:

Let's consider a variable b.

b + +

Here, b = b + 1. It is called the increment operator.

b + =

For example, b + = 4. It means, b = b+ 4.

b - -

Here, b = b - 1. It is called as the decrement operator.

b - =

For example, b - = 3. It means, b = b - 3.

b * =

For example, b * = 6. It means, b = b * 6.

b / =

For example, b / = 5. It means, b = b / 5.

b % =

For example, b % = 2. It means, b = b % 2.

Now, let's use the above operators with two variables, b and c.

b + = c (b = b + c)

b - = c (b = b - c)

b * = c (b = b * c)

b / = c (b = b / c)

b % = c (b = b % c)

We can specify any variable instead of b and c.

3.Boolean Operators

The Boolean Operators are NOT (!), Logical AND (& &), and Logical OR (| |).

Let's discuss the above operators in detail.

- Logical AND (& &)

The result of the condition is true if both the operands in the condition are true.

Consider the below example:

if (a = = b & & b = = c)

Above statement is true if both conditions are true. If any of the conditions is false, the statement will be false.

- Logical OR (| |)

The result of the condition is true, if either of the variables in the condition is true.

Consider the below example.

if (a > 0 | | b > 0)

The above statement is true, if either of the above condition (a> 0 or b > 0) is true.

- NOT (!)

It is used to reverse the logical state of the operand.

For example, a ! = 2.

The NOT operator returns the value 1 or TRUE when the specified operand is FALSE. It also reverses the value of the specified expression.

4.Comparison Operators

The comparison operators are used to compare the value of one variable with the other.

The comparison operators are listed below:

- less than (<)

The less than operator checks that the value of the left operand is less than the right operand. The statement is true if the condition is satisfied.

Consider the below code.

```
int b;

int c ;

void setup ( )

{
```

```
Serial.begin( 9600 );

}

void loop ( )

{

b = 3;

c = 5;

if ( b < 4 )

Serial.println(b);

if ( c < 4)

Serial.println( c);

}
```

Output: 3

In the above code, if any of the two statement is correct, the corresponding value of the variable will be printed. Here, only first condition is correct. Hence, the value of b will be printed.

- greater than (>)

The less than operator checks that the value of the left side of a statement is greater than the right side. The statement is true if the condition is satisfied.

For example, a > b.

If a is greater than b, the condition is true, else false.

- equal to (= =)

It checks the value of two operands. If the values are equal, the condition is satisfied.

For example, a = = b.

The above statement is used to check if the value of a is equal to b or not.

- not equal to (! =)

It checks the value of two specified variables. If the values are not equal, the condition will be correct and satisfied.

For example, a ! = b.

less than or equal to (< =)

The less or equal than operator checks that the value of left side of a statement is less or equal to the value on right side. The statement is true if either of the condition is satisfied.

For example, a < = b

It checks the value of a is less or equal than b.

- greater than or equal to (> =)

The greater or equal than operator checks that the value of the left side of a statement is greater or equal to the value on the right side of that statement. The statement is true if the condition is satisfied.

For example, a > = b

It checks the value of a is greater or equal than b. If either of the condition satisfies, the statement is true.

5.Bitwise Operators

The Bitwise operators operate at the binary level. These operators are quite easy to use.

There are various bitwise operators. Some of the popular operators are listed below:

- bitwise NOT (~)

The bitwise NOT operator acts as a complement for reversing the bits.

For example, if b = 1, the NOT operator will make the value of b = 0.

Let's understand with another example.

0 0 1 1 // Input or operand 1 (decimal value 3)

1 1 0 0 // Output (reverses the input bits) decimal value is 12

- bitwise XOR (^)

The output is 0 if both the inputs are same, and it is 1 if the two input bits are different.

For example:

1 0 0 1 // input 1 or operand 1

0 1 0 1 // input 2

1 1 0 0 // Output (resultant - XOR)

- bitwise OR (|)

The output is 0 if both of the inputs in the OR operation are 0. Otherwise, the output is 1. The two input patterns are of 4 bits.

For example:

1 1 0 0 // input 1 or operand 1

0 0 0 1 // input 2

1 1 0 1 // Output (resultant - OR)

- bitwise AND (&)

The output is 1 if both the inputs in the AND operation are 1. Otherwise, the output is 0. The two input patterns are of 4 bits.

For example:

1 1 0 0 // input 1 or operand 1

0 1 0 1 // input 2

0 1 0 0 // Output (resultant - AND)

- bitwise left shift (< <)

The left operator is shifted by the number of bits defined by the right operator.

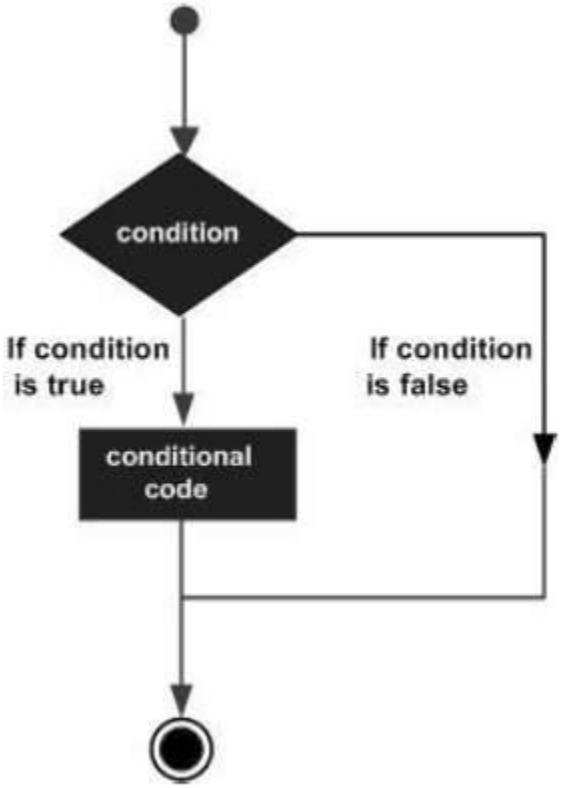
- bitwise right shift (> >)

The right operator is shifted by the number of bits defined by the left operator.

❖ 3.5 Control statements and Loops

Decision making structures require that the programmer specify one or more conditions to be evaluated or tested by the program. It should be along with a statement or statements to be executed if the condition is determined to be true, and optionally, other statements to be executed if the condition is determined to be false.

Following is the general form of a typical decision making structure found in most of the programming languages –

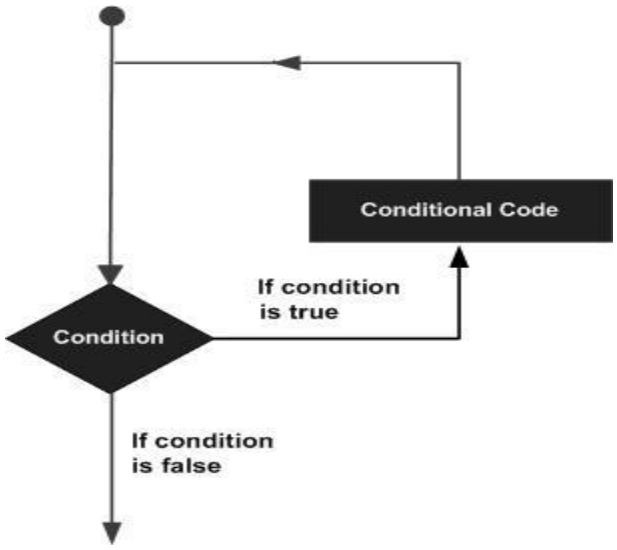


.NO.	Control Statement & Description
1	**If statement** It takes an expression in parenthesis and a statement or block of statements. If the expression is true then the statement or block of statements gets executed otherwise these statements are skipped.
2	**If …else statement** An **if** statement can be followed by an optional else statement, which executes when the expression is false.
3	**If…else if …else statement** The **if** statement can be followed by an optional **else if...else** statement, which is very useful to test various conditions using single if...else if statement.
4	**switch case statement** Similar to the if statements, **switch...case** controls the flow of programs by allowing the programmers to specify different codes that should be executed in various conditions.

5	**Conditional Operator ? :** The conditional operator ? : is the only ternary operator in C.

Programming languages provide various control structures that allow for more complicated execution paths.

A loop statement allows us to execute a statement or group of statements multiple times and following is the general form of a loop statement in most of the programming languages −



S.NO.	Loop & Description
1	**while loop** while loops will loop continuously, and infinitely, until the expression inside the parenthesis, () becomes false. Something must change the tested variable, or the while loop will never exit.
2	**do…while loop** The **do…while** loop is similar to the while loop. In the while loop, the loop-continuation condition is tested at the beginning of the loop before performed the body of the loop.
3	**for loop** A **for loop** executes statements a predetermined number of times. The control expression for the loop is initialized, tested and manipulated entirely within the for loop parentheses.
4	**Nested Loop** C language allows you to use one loop inside another loop. The following

	example illustrates the concept.
5	**Infinite loop** It is the loop having no terminating condition, so the loop becomes infinite.

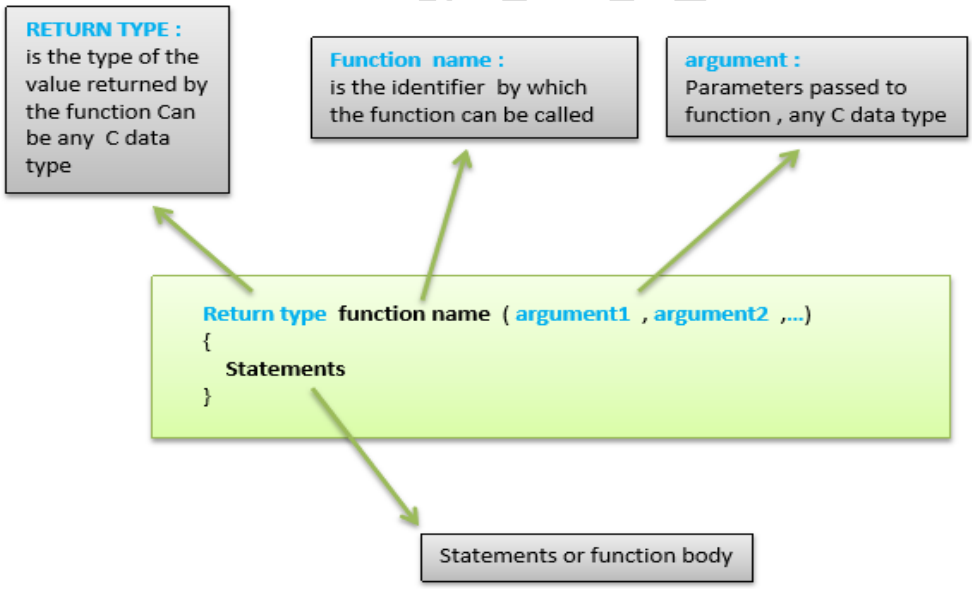
❖ 3.6 Functions and library functions

- User defined functions

  In Arduino programming, user-defined functions are custom functions created by the user to perform specific tasks. These functions help in organizing code, making it more readable, and facilitating code reuse. Here's a basic structure of how we can create user-defined functions in Arduino:

  There are two required functions in an Arduino sketch or a program i.e. setup () and loop(). Other functions must be created outside the brackets of these two functions:

  The most common syntax to define a function is:-



The most common syntax to define a function is

- **Example**

// Function declaration

// Return_type function_name(parameter_type parameter_name);

// Example function declaration

```
int addNumbers(int a, int b);

void setup() {

  // Setup code goes here

  Serial.begin(9600);

  // Function call

  int result = addNumbers(5, 7);

  // Print the result

  Serial.println(result);

}

void loop() {

  // Main loop code goes here

}

// Example function definition

int addNumbers(int a, int b) {

  int sum = a + b;

  return sum;

}
```

- **Explanation:**

Function Declaration:

A function declaration provides information about the function, including its return type, name, and parameter types.

It serves as a prototype for the function, letting the compiler know what to expect when the function is defined later in the code.

Function Definition:

The function definition includes the actual implementation of the function.

It specifies the return type, function name, and parameters, followed by the code block that contains the function's logic.

In the example, addNumbers is a function that takes two integers as parameters and returns their sum.

Function Call:

In the setup function (which runs once at the beginning), we call the addNumbers function with arguments 5 and 7.

The result is stored in a variable, and then it is printed using the Serial.println function.

- Library functions

  - **I/O Functions**

In Arduino programming, I/O (Input/Output) functions are used to interact with digital and analog pins to read inputs or control outputs. These functions are essential for working with sensors, actuators, and other peripherals connected to the Arduino board. Here are some commonly used I/O functions in Arduino:

Digital I/O Functions:

1.pinMode(pin, mode):

Configures a specified digital pin to be either an input or an output.

Example: pinMode(13, OUTPUT);

2.digitalWrite(pin, value):

Writes a HIGH or LOW value to a specified digital pin.

Example: digitalWrite(8, HIGH);

3.digitalRead(pin):

Reads the digital value (HIGH or LOW) from a specified digital pin.

Example: int sensorValue = digitalRead(2);

Analog I/O Functions:

1.analogRead(pin):

Reads the analog voltage value from a specified analog pin (0 to 1023).

Example: int analogValue = analogRead(A0);

2.analogWrite(pin, value):

Writes a PWM (Pulse Width Modulation) value to a specified PWM-enabled pin (commonly used for simulating analog output).

Example: analogWrite(9, 128);

Time Functions:

1.delay(ms):

Pauses the program execution for the specified number of milliseconds.

Example: delay(1000); (pauses for 1 second)

2.millis():

Returns the number of milliseconds since the Arduino board began running the current program.

Useful for creating non-blocking delays.

Example: unsigned long currentTime = millis();

3.micros():

Returns the number of microseconds since the Arduino board began running the current program.

Example: unsigned long elapsedTime = micros();

Serial Communication Functions:

1.Serial.begin(baudrate) and Serial.end():

Initializes serial communication with a specified baud rate.

Example: Serial.begin(9600);

2.Serial.print(value) and Serial.println(value):

Prints data to the serial port.

Example: Serial.print("Sensor value: "); Serial.println(sensorValue);

3.Serial.available() and Serial.read():

Checks for available data in the serial buffer and reads a byte.

Example: if (Serial.available() > 0) { char data = Serial.read();
}
These functions form the foundation for interacting with the input and output capabilities of Arduino boards. By using these functions, you can read from sensors, control actuators, and communicate with other devices. It's important to refer to the Arduino documentation for more details on each function and their usage.

- **char Data Type:**

In Arduino programming, character (char) functions are used for working with individual characters or sequences of characters (strings). Here are some commonly used char functions and techniques in Arduino:

1. Char Data Type:

The char data type in Arduino is used to store a single character (8 bits).

char myChar = 'A';

2. String Initialization and Manipulation:

Arduino supports the use of the String class for working with strings.

String myString = "Hello, Arduino!";

Common String functions include:

length(): Returns the number of characters in the String.

charAt(index): Returns the character at the specified index.

concat(str): Concatenates (appends) another string to the end.

substring(start, length): Returns a substring starting from the specified index and of the specified length.

3. Converting Numbers to Strings:

You can convert numerical values to strings using the String() constructor or the dtostrf() function for floating-point numbers.

int myNumber = 42;

String myNumberString = String(myNumber);

4. Converting Strings to Numbers:

You can convert strings to numerical values using functions like toInt(), toFloat(), toDouble(), etc.

```
String numString = "123";

int num = numString.toInt();
```

5. Character Array (C-String):

In Arduino, you often work with character arrays (C-strings) when dealing with strings.

```
char myCharArray[] = "Arduino";
```

Common char array functions include:

strlen(charArray): Returns the length of the C-string.

strcmp(str1, str2): Compares two C-strings.

strcat(dest, src): Concatenates (appends) one C-string to another.

6. Serial Communication with Characters:

When working with serial communication, you often use Serial.read() to read individual characters from the serial buffer.

```
char receivedChar = Serial.read();
```

These are just a few examples of working with characters and strings in Arduino. When working with strings, especially if you need dynamic memory allocation, be mindful of the memory constraints of Arduino boards. Additionally, using C-strings and the String class should be chosen based on the requirements and constraints of your specific application.

- **Math Functions**

1. abs(x):

Returns the absolute value of x (the distance from zero without regard to the sign).

Example: int result = abs(-5); // result is 5

2. sqrt(x):

Returns the square root of x.

Example: float value = 9; float squareRootValue = sqrt(value); // squareRootValue is 3

3. pow (base, exponent):

Returns the value of base raised to the power of exponent.

Example: float result = pow(2, 3); // result is 8

4. random (min, max):

Returns a random number between min and max. The randomSeed(seed) function can be used to initialize the random number generator.

Example: int randomNumber = random(0, 10); // random number between 0 and 9

5. constrain()

```
constrain(value, lowerLimit, upperLimit);
```

value: The value you want to constrain.

lowerLimit: The lower limit or minimum allowed value.

upperLimit: The upper limit or maximum allowed value.

The function returns the constrained value within the specified range.

❖ LED Blinking using Arduino

Blinking an LED is one of the simplest projects you can do with an Arduino. It's often used as a basic introductory example to demonstrate how to set up the Arduino, write a simple program, and control an output. Here's a step-by-step guide to blink an LED using an Arduino:

- Hardware Setup:

1.Components Needed:

- Arduino board (e.g., Arduino Uno)
- LED (any color)
- Resistor (around 220-470 ohms)
- Jumper wires

2.Circuit Connections:

- Connect the longer leg (anode) of the LED to a digital pin on the Arduino (e.g., pin 13).
- Connect the shorter leg (cathode) of the LED to the ground (GND) pin on the Arduino.
- Connect one end of the resistor to the same digital pin to which the LED is connected.
- Connect the other end of the resistor to the positive voltage (5V) on the Arduino.

Arduino Code

```
// Define the pin to which the LED is connected

const int ledPin = 13;

void setup() {

  // Set the LED pin as an output

  pinMode(ledPin, OUTPUT);

}

void loop() {

  // Turn on the LED

  digitalWrite(ledPin, HIGH);

  // Wait for 1 second

  delay(1000);

  // Turn off the LED

  digitalWrite(ledPin, LOW);

  // Wait for 1 second

  delay(1000);

}
```

Explanation:

1. const int ledPin = 13;:

- Defines a constant variable ledPin and assigns it the value 13. This is the digital pin to which the LED is connected.

2. pinMode(ledPin, OUTPUT);:

- Configures the ledPin as an output. This step is done in the setup() function, which runs once at the beginning.

3. digitalWrite(ledPin, HIGH);:

- Turns on the LED by setting the voltage on the ledPin to HIGH (5V).

4. delay(1000);:

- Pauses the program for 1 second (1000 milliseconds).

5. digitalWrite(ledPin, LOW);:

- Turns off the LED by setting the voltage on the ledPin to LOW (0V).

6. Looping:

- The loop() function runs continuously, causing the LED to blink on and off in a 1-second interval.

Uploading the Code:

1. Connect your Arduino to your computer using a USB cable.
2. Open the Arduino IDE
3. Copy and paste the code into the IDE.
4. Select your Arduino board from the "Tools" > "Board" menu.
5. Select the appropriate port from the "Tools" > "Port" menu.
6. Click the "Upload" button to upload the code to the Arduino..

Once the code is uploaded, you should see the LED connected to pin 13 on the Arduino blinking on and off at a 1-second interval.

❖ Serial Communication Functions

Serial communication is a crucial aspect of Arduino programming, especially for debugging, monitoring, and interacting with other devices. Here are some essential serial communication functions in Arduino:

1. **Serial.begin(baudrate):**

   Initializes serial communication with a specified baud rate. Common baud rates include 9600, 115200, etc.

   Example: Serial.begin(9600);

2. **Serial.available():**

   Returns the number of bytes available in the serial buffer.

   Example: int bytesAvailable = Serial.available();

3. **Serial.println (data):**

Prints data to the serial port followed by a newline character ('\n').

Example: Serial.println("Hello, Arduino!");

4.Serial.print(data):

Prints data to the serial port without adding a newline character.

Example: int sensorValue = analogRead(A0); Serial.print("Sensor Value: ");
Serial.println(sensorValue);

5.Serial.read():

Reads a byte of data from the serial buffer. Returns -1 if no data is available.

Example: char receivedChar = Serial.read();

7.Serial.write(data):

Writes a byte of data to the serial port.

Example: Serial.write(65); // Writes the ASCII character 'A'

8.Serial.readBytes()

Description

Serial.readBytes() reads characters from the serial port into a buffer. The function terminates if the determined length has been read, or it times out (see Serial.setTimeout()).

Serial.readBytes() returns the number of characters placed in the buffer. A 0 means no valid data was found.

Serial.readBytes() inherits from the Stream utility class.

Syntax

Serial.readBytes(buffer, length)

9.Serial.end()

Description

Disables serial communication, allowing the RX and TX pins to be used for general input and output. To re-enable serial communication, call Serial.begin().

Syntax

Serial.end()

10. Serial.readString()

Description

Serial.readString() reads characters from the serial buffer into a String. The function terminates if it times out (see setTimeout()).

Serial.readString() inherits from the Stream utility class.

Syntax

Serial.readString()

VPMP POLYTECHNIC